

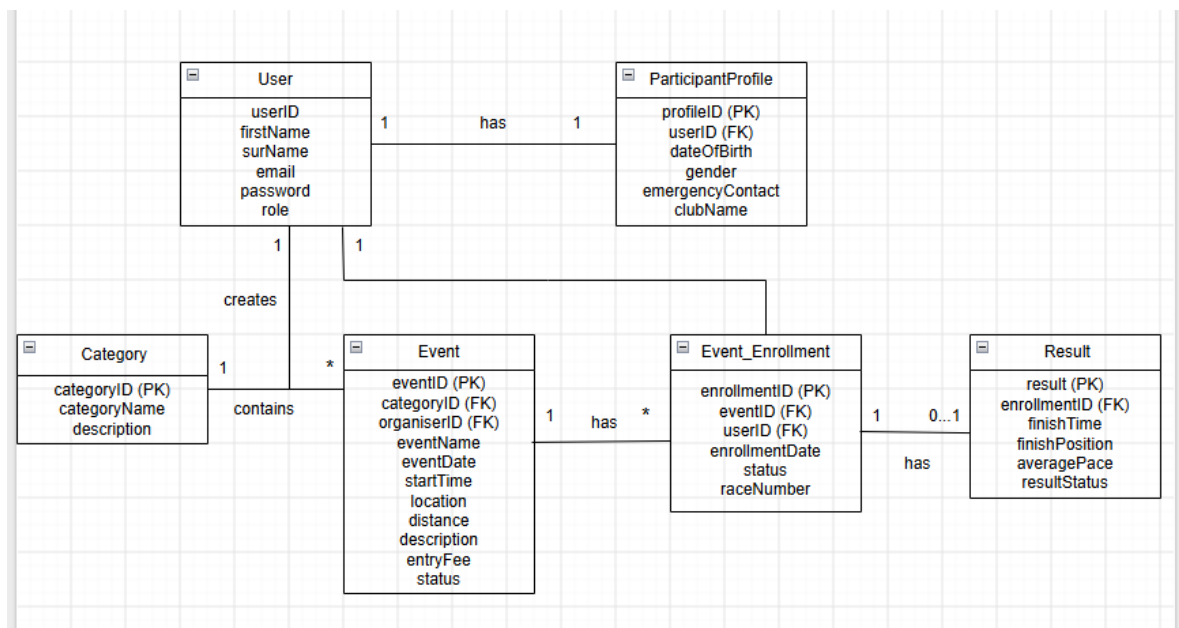
RETSHEGO MOSHE

ST10475142

PROGRAMMING 2B

RaceDay – API Endpoint Plan

Part 1 Section A



The ERD is designed for the RaceDay Event management system. The whole purpose of this is to display how the main data that is inside the system is put together and how the different entities are related to one another.

The user entity keeps the basic information of people who use the RaceDay system, the ParticipantProfile entity keeps the added information that is specifically about the participant, the Category entity keeps the different categories in which RaceDay events are organised, the Event entity holds the main information about RaceDay events, the Event_Enrollment entity documents participants who enrol in the RaceDay events, the Result entity is keeps the results achieved by participants after completing an event.

Part 1 Section B

This document serves the purpose of defining REST API endpoints that is planned for the RaceDay event management system. The endpoint is backed up by authentication, user profiles, events, categories, event enrolment and participant results.

HTT P Met hod	Route	Descri ption	Role Requir ed	Request Body (If any)	Expect ed Respo nse
POST	/api/auth/register	Register a new RaceDay user account.	Public	FirstName, Surname, Email, Password, Role	201 Created – User account created successfully.
POST	/api/auth/login	Authenticates a user using their email and password.	Public	Email, Password	200 OK – Authentication token and user information.
GET	/api/users/{id}	Retrieves a user's profile information.	Authenticated user	None	200 OK – User profile details.
PUT	/api/users/{id}	Updates a user's profile information.	User	FirstName, Surname, Email	200 OK – Updated user profile.
GET	/api/users/{id}/profile	Retrieves the participant's profile information.	Authenticated user	None	200 OK – Participant profile details.
GET	/api/events	Retrieves all available events.	Public	None	200 OK – list of events.
GET	/api/categories	Retrieves all event categories.	Public	None	200 OK – list of categories.

POST	/api/events/{eventid}/enrolments	Enrols a participant in an event	Participant	UserID	201 Created – enrolment created successfully.
GET	/api/events/{eventid}/results	Retrieves the results for a specific event.	Public	None	200 OK – list of event results.
GET	/api/events/{eventId}	Retrieves details of a specific event.	Public	None	200 OK – Event details.
POST	/api/events	Creates a new RaceDay	Organiser	categoryID, eventName, startTime, Location, Distance, Description, entryFee,status	200 OK – Event updated successfully.
PUT	/api/events/{eventID}	Updates the details of an existing event.	Organiser	eventName,eventDate, startTime, Location, Distance,Description, entryFee, status	200 OK – Event updated successfully.
DELETE	/api/events/{eventID}	Removes an event from the system	organiser	None	200 OK- Event deleted successfully
GET	/api/categories/{categoryID}	Retrieves details of a specific event category	Public	None	200 OK – Category details
POST	/api/categories	Creates a new event	Organiser	CategoryName,Description	201 Created –

		category			Category created successfully
GET	/api/events/{eventId}/enrolments	Retrieves all participants enrolled in a specific	Organiser	None	200 OK – List of enrolments
DELETE	/api/events/{eventId}/enrolment/{enrollmentId}	Cancels a participant's enrolment in an event	Participant	None	200 OK – Enrolment cancelled successfully
POST	/api/events/{eventId}/results	Records a participant result for an event	Organiser	EnrollmentID, FinishTime, FinishPosition, AveragePace, ResultStatus	201 Created - Result recorded successfully
PUT	/api/events/{eventId}/result/{resultId}	Updates an existing participant result	Organiser	FinishTime, FinishPosition, AveragePace, ResultStatus	200 OK – Result updated successfully

Part 1 Section C

```

1 CREATE DATABASE RACEDAYdb;
2
3 CREATE TABLE [User]
4 (
5     userID INT PRIMARY KEY, [
6     firstName VARCHAR(50) NOT NULL,
7     surName VARCHAR(100) NOT NULL,
8     email VARCHAR(100) NOT NULL UNIQUE,
9     password VARCHAR(255) NOT NULL,
10    role VARCHAR (20) NOT NULL DEFAULT 'Participant'
11    CHECK (role IN ( 'Participant', 'Organiser'))
12 );

```

This code makes the User table to store account information. userID is the primary key, UNIQUE prevents duplicate email addresses, the role is to identify whether the user is a Participant or Organiser.

	userID	firstName	surName	email	password	role
1	1	Thabo	Mokoena	thabo.mokoena@raceday.co.za	Thabo@123	Organiser
2	2	Naledi	Dlamini	naledi.dlamini@raceday.co.za	Naledi@123	Organiser
3	3	Lerato	Molefe	lerato.molefe@gmail.com	Lerato@123	Participant
4	4	Sipho	Nkosi	sipho.nkosi@gmail.com	Sipho@123	Participant

The INSERT adds four sample users to the database. It illustrates the two required roles with the 2 Organisers and 2 Participants.

```
CREATE TABLE ParticipantProfile
(
  profileID INT PRIMARY KEY,
  userID INT NOT NULL UNIQUE,
  dateOfBirth DATE NOT NULL,
  gender VARCHAR(20) NOT NULL,
  emergencyContact VARCHAR(100) NOT NULL,
  clubName VARCHAR(100),

  CONSTRAINT FK_ParticipantProfile_User
  FOREIGN KEY (userID)
  REFERENCES [User](userID)
);
```

The ParticipantProfile table keeps added information about the participants, such as their dates of birth, gender, and emergency.

profileID identifies each profile userID connects the profile to the User table. The UNIQUE constraints make sure that one user cannot have multiple participant profile.

	profileID	userID	dateOfBirth	gender	emergencyContact	clubName
1	10	3	2004-06-15	Female	0712345678	Johannesburg Striders
2	20	4	2003-11-22	Male	0723456789	Soweto Runners Club

The INSERT adds profile information for the 2 participants. The userID connects each profile to the correct user.

```
CREATE TABLE Category
(
  categoryID INT PRIMARY KEY,
  categoryName VARCHAR(100) NOT NULL,
  description VARCHAR(255)
);
```

The category table stores the different types of events offered by RaceDay.

categoryID is the primary key and identifies each category. categoryName keeps the category name, while description provides additional information about it.

	categoryID	categoryName	description
1	30	Road Running	Running events held on public roads and urban ro...
2	60	Walking	Community and recreational walking events.
3	90	Cycling	Road cycling events for recreational and competi...

The INSERT adds three categories: Road Running, Walking and Cycling. These categories can then be linked to event.

```
CREATE TABLE Event
(
    eventID INT PRIMARY KEY,
    categoryID INT NOT NULL,
    organiserID INT NOT NULL,
    eventName VARCHAR(100) NOT NULL,
    eventDate DATE NOT NULL,
    startTime TIME NOT NULL,
    location VARCHAR(100) NOT NULL,
    distance VARCHAR(20) NOT NULL,
    description VARCHAR(500),
    entryFee DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    status VARCHAR(20) NOT NULL DEFAULT 'Open',

    CONSTRAINT FK_Event_Category
        FOREIGN KEY (categoryID)
        REFERENCES Category(categoryID),

    CONSTRAINT FK_Event_Organiser
        FOREIGN KEY (organiserID)
```

The Event table keeps information about RaceDay events, including the event name, date, location, distance and entry fee.

eventID identifies each event. categoryID links the event to a category, while organiserID identifies the organiser. The remaining fields keep the event's detail, including its date, time, location, distance, fee and status.

	eventID	categoryID	organiserID	eventName	eventDate	startTime	location	distance	description	entryFee	status
1	21	30	1	Johannesburg City Run	2026-10-18	07:00:00.0000000	Johannesburg	10 km	A city road-running event for recreational and com...	150.00	Open
2	22	60	2	Soweto Community Walk	2026-11-08	08:00:00.0000000	Soweto	5 km	A community walking event promoting health and a...	80.00	Open
3	23	90	1	Cape Town Cycle Challenge	2026-11-22	06:30:00.0000000	Cape Town	40 km	A road cycling event for cyclists of different experi...	250.00	Open

The INSERT adds to a category and

an organiser using their IDs. The information illustrates how real RaceDay events can be stored.

```
CREATE TABLE Event_Enrollment (
    enrollmentID INT IDENTITY(1,1) PRIMARY KEY,
    eventID INT NOT NULL,
    userID INT NOT NULL,
    enrollmentDate DATE NOT NULL,
    status VARCHAR(30) NOT NULL,
    raceNumber VARCHAR(20) NOT NULL,

    FOREIGN KEY (eventID) REFERENCES Event(eventID),
    FOREIGN KEY (userID) REFERENCES [User](userID)
);
```

The Event_Enrollment table records participants who register for RaceDay events.

enrollmentID identifies each enrolment. eventID identifies the event, while userID identifies the participant. This table connects participants with events and therefore resolves the many – to – many relationships.

	enrollmentID	eventID	userID	enrollmentDate	status	raceNumber
1	9	21	3	2026-09-02	Confirmed	RUN001
2	10	22	4	2026-09-02	Confirmed	RUN002
3	11	21	3	2026-09-02	Confirmed	WALK001
4	12	23	4	2026-09-02	Confirmed	CYCLE001
5	13	21	3	2026-09-02	Confirmed	RUN001
6	14	22	4	2026-09-02	Confirmed	RUN002
7	15	21	3	2026-09-02	Confirmed	WALK001
8	16	23	4	2026-09-02	Confirmed	CYCLE001

The INSERT records four event enrolments. It shows which participant registered for which events and stores their enrolment date, status and the race number.

```
CREATE TABLE Result
(
    result INT PRIMARY KEY,
    enrolmentID INT NOT NULL,
    finishTime TIME,
    finishPosition INT,
    averagePace DECIMAL(10,2),
    resultStatus VARCHAR(50),

    FOREIGN KEY(enrolmentID)
    REFERENCES Event_Enrollment(enrollmentID)
);
```

The Result table stores the results achieved by participants after completing an event.

The result field is the primary key and identifies each result. enrolmentID connects the fields store the participant's finishing time, position, averagePace and result status.